DESARROLLO DE SISTEMAS INFORMÁTICOS

Sistema de Gestión de Incidentes (Help Desk) – Estudio de Caso

LINK GITHUB

https://github.com/leoriosmoreno/DesarrolloSistemasInformaticosUTM.git

RIOS MORENO LUIS LEONARDO

Docente: Ing. Jorge Arun Zambrano Cedeño, Mg.

(fecha de cierre de la actividad 16,05,2026)

Actividad 1 a desarrollar en la Unidad 1

ARQUITECTURA LÓGICA DEL SISTEMA DE GESTIÓN DE INCIDENTES (HELP DESK) EN UN DATA CENTER - Informe Técnico

ANÁLISIS DEL PROBLEMA: ESCENARIO DE DATA CENTER SIMULADO

En un centro de datos, la gestión de fallos debe ser inmediata para evitar interrupciones críticas. El sistema diseñado actúa como un punto único de contacto (SPOC) que centraliza alertas de sensores y reportes humanos, asegurando que cada incidente sea atendido por el especialista correcto mediante una estructura de niveles de soporte (Nivel 1, 2 y 3).

Identificación de Actores

Para cubrir todas las interacciones con la infraestructura, se han definido cinco actores clave:

1. **Usuario Final / Cliente:** Reporta fallos en servicios y consulta el estado de sus solicitudes.
2. **Técnico de Soporte (Nivel 1/2):** Diagnostica, resuelve incidentes comunes y actualiza bitácoras.
3. **Administrador de Sistemas (Nivel 3):** Especialista en infraestructura crítica que interviene en escalamientos complejos.
4. **Administrador del Help Desk:** Gestiona la configuración, niveles de servicio (SLA) y métricas.
5. **Sistema de Monitorización (Actor Externo):** Sensores y agentes de red que generan alertas automáticas sin intervención humana.

Requerimientos de Software

<u>Requerimientos Funcionales (RF)</u>

1. **Registro y Categorización:** Creación manual o automática de tickets clasificados en Hardware, Red o Software.
2. **Gestión de Ciclo de Vida:** Los incidentes deben transitar por estados: Abierto, En Análisis, Pendiente, Escalado, Resuelto y Cerrado.
3. **Sistema de Alertas Reactivo:** Notificaciones push/correo inmediatas ante cambios de prioridad a “Crítico”.
4. **Asignación Experta:** El motor debe vincular el ticket al técnico disponible con la especialidad adecuada.

5. **Escalamiento Automático:** Si un ticket no se resuelve en un tiempo definido, el sistema debe transferir la propiedad a un nivel superior.

Requerimientos No Funcionales (RNF)

1. **Alta Disponibilidad:** Operación continua del 99.9% del tiempo (Uptime).
2. **Seguridad y Trazabilidad:** Historial inalterable de cambios y acceso cifrado basado en roles.
3. **Escalabilidad Horizontal:** Capacidad de procesar miles de alertas de sensores IoT simultáneamente.
4. **Control de Versiones:** Uso mandatorio de Gitflow para la integridad del ciclo de desarrollo.

DISEÑO DEL SISTEMA MEDIANTE MODELADO UML

Diagrama de Casos de Uso

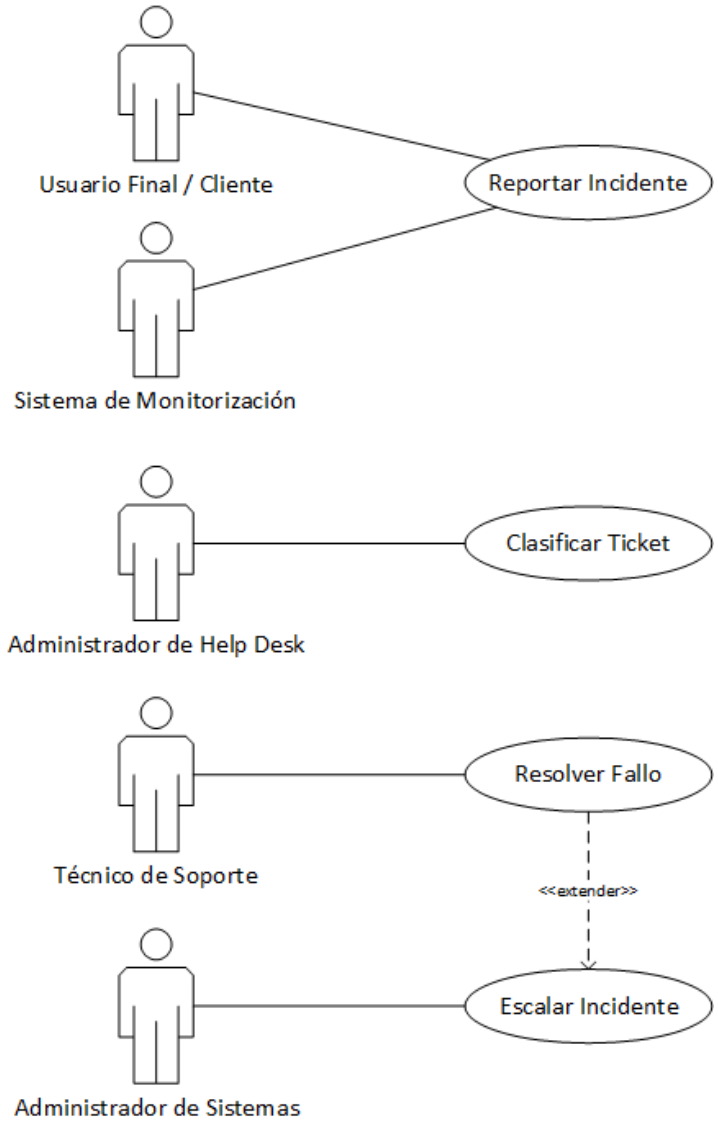
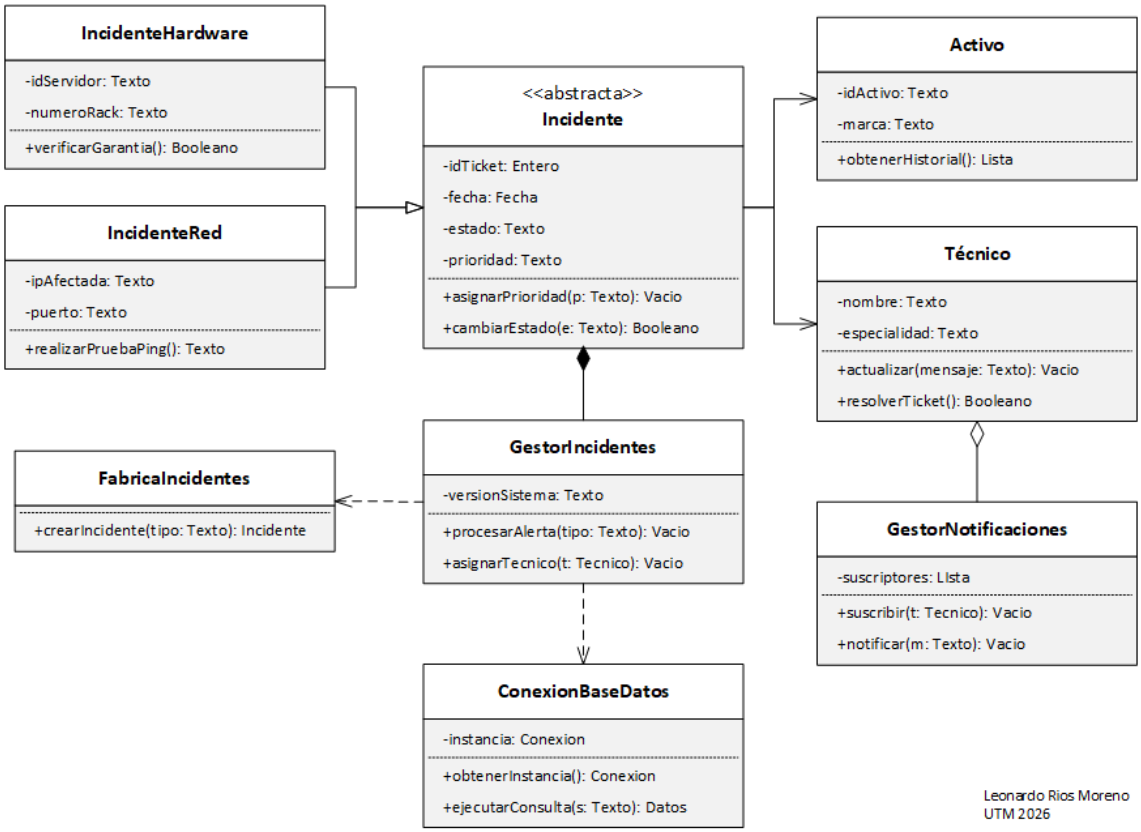
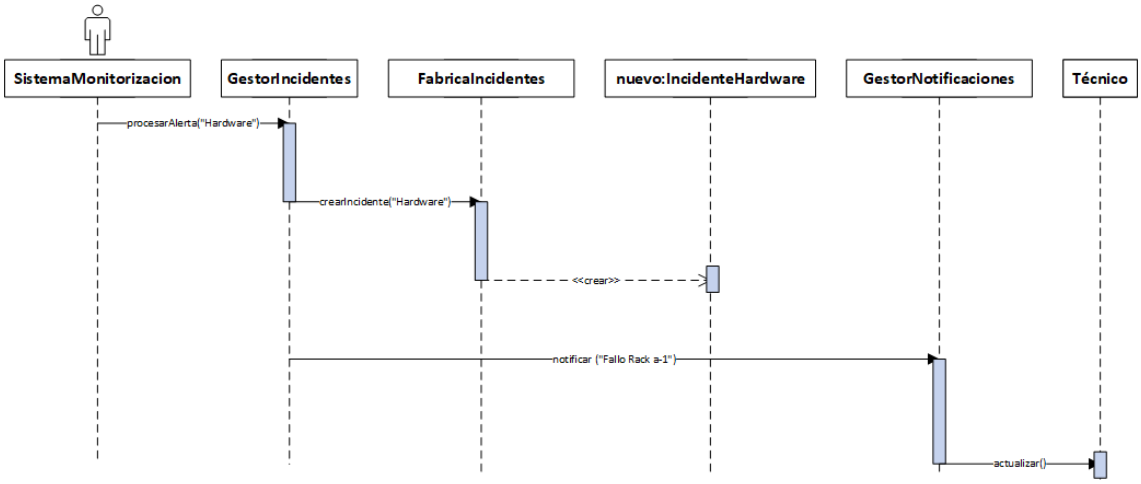


Diagrama de Clases UML y Principios SOLID



Principio	Aplicación en el Help Desk	Beneficio en el Data Center
Responsabilidad Única (SRP)	ConexionBaseDatos solo maneja persistencia, mientras que GestorNotificaciones gestiona alertas.	Facilita el mantenimiento al localizar errores en módulos específicos sin afectar el resto del sistema.
Abierto/Cerrado (OCP)	La jerarquía de Incidente permite añadir IncidenteSoftware extendiendo la clase base sin modificar el motor central.	Permite la evolución tecnológica del centro de datos sin riesgos de regresión en el código base.
Sustitución de Liskov (LSP)	El GestorIncidentes puede procesar cualquier objeto Incidente sea de tipo Hardware o Red de forma genérica.	Garantiza que las operaciones genéricas de gestión de tickets funcionen para cualquier especialidad técnica.
Segregación de Interfaz (ISP)	Se definen interfaces como IAccionesTecnico e IConfiguracionAdmin en lugar de una interfaz monolítica.	Evita que los técnicos tengan acceso a métodos de configuración que no les corresponden por rol.
Inversión de Dependencias (DIP)	El motor depende de la abstracción Incidente y no de las implementaciones de bajo nivel.	Permite cambiar el motor de base de datos o migrar a la nube con un impacto mínimo en la lógica de negocio.

Diagrama de Secuencia



Arquitectura General y Flujo de Versiones (Gitflow)

Significa que la estructura lógica del software (Capas de Presentación, Negocio y Datos) se alinea con el ciclo de vida del repositorio en GitHub. Cada avance en el diseño de un patrón o diagrama se registra en una rama feature/ específica, garantizando que el diseño final en la rama master sea una arquitectura validada y sin errores.

Para el registro de avances en el repositorio GitHub, se utiliza el modelo Gitflow para separar claramente el estado estable del desarrollo experimental:

- **Master:** Código de arquitectura final y aprobado.
- **Develop:** Integración de componentes de diseño.
- **Feature Branches:** feature/uml-modeling, feature/design-patterns, permitiendo trabajar en módulos aislados sin afectar la estabilidad principal.

APLICACIÓN DE PATRONES Y PARADIGMA DE DISEÑO

Se han implementado tres patrones clave para garantizar la flexibilidad y reactividad de la arquitectura.

Patrón	Propósito Técnico	Aplicación en el Help Desk
Singleton	Garantiza una única instancia de un objeto y punto de acceso global.	Utilizado para la Conexión a Base de Datos y el Motor de Numeración de Tickets, evitando colisiones de datos.
Observer	Define un mecanismo de suscripción para notificar cambios de estado.	Los técnicos se suscriben como “observadores” a categorías de tickets. Al cambiar un estado a “Crítico”, todos son notificados automáticamente vía push/email.
Factory Method	Delega la creación de objetos a subclases según una interfaz común.	Se usa para crear objetos Incident de forma dinámica (Hardware, Red, Software) sin que el sistema principal conozca la clase concreta al momento de la creación.

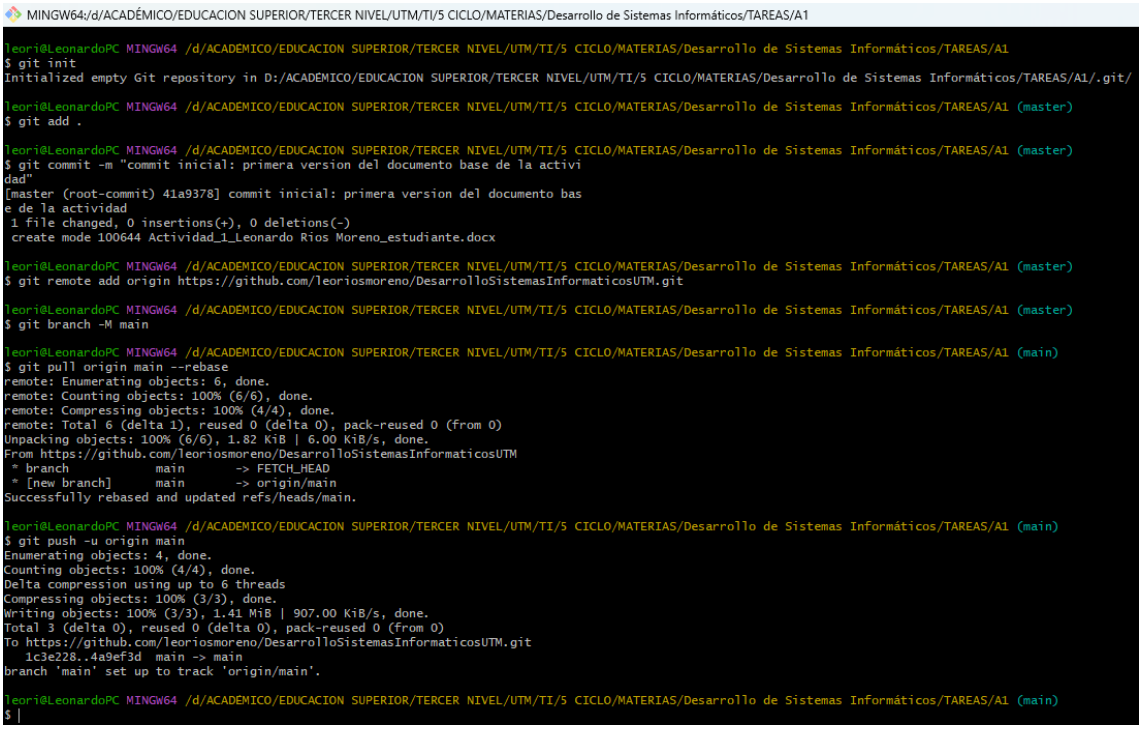
Paradigma Seleccionado

Se ha seleccionado un **Diseño Orientado a Objetos (DOO)** combinado con un **Diseño Orientado a Eventos (DOE)**.

- **Justificación:** El DOO permite modelar los activos físicos del Data Center como objetos con atributos precisos. El DOE es indispensable para manejar las alertas asíncronas de los sensores de infraestructura, permitiendo que el sistema reaccione sin bloquear otros procesos del personal de soporte.

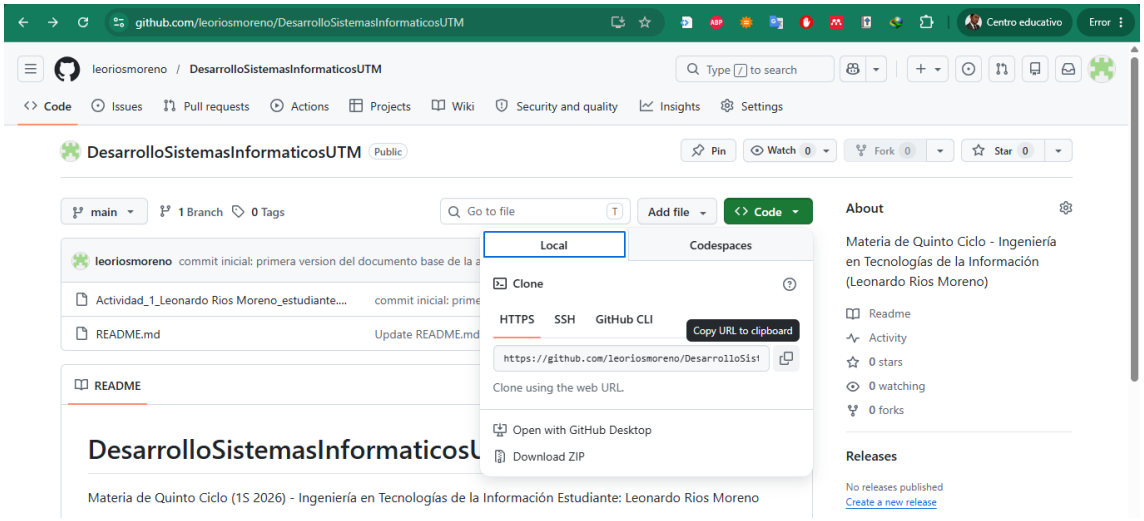
Evidencias

PROCESO GIT (Local)



```
MINGW64:/d/ACADÉMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1

leori@LeonardoPC MINGW64 /d/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1
$ git init
Initialized empty Git repository in D:/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1/.git/

leori@LeonardoPC MINGW64 /d/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1 (master)
$ git add .

leori@LeonardoPC MINGW64 /d/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1 (master)
$ git commit -m "commit inicial: primera version del documento base de la activi
dad"
[master (root-commit) 41a9378] commit inicial: primera version del documento bas
e de la actividad
 1 file changed, 0 insertions(+), 0 deletions(-)
 create mode 100644 Actividad_1_Leonardo Rios Moreno_estudiante.docx

leori@LeonardoPC MINGW64 /d/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1 (master)
$ git remote add origin https://github.com/leoriosmoreno/DesarrolloSistemasInformaticosUTM.git

leori@LeonardoPC MINGW64 /d/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1 (master)
$ git branch -M main

leori@LeonardoPC MINGW64 /d/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1 (main)
$ git pull origin main --rebase
remote: Enumerating objects: 6, done.
remote: Counting objects: 100% (6/6), done.
remote: Compressing objects: 100% (4/4), done.
remote: Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (6/6), 1.82 KiB | 6.00 KiB/s, done.
From https://github.com/leoriosmoreno/DesarrolloSistemasInformaticosUTM
 * branch            main       -> FETCH_HEAD
 * [new branch]      main       -> origin/main
Successfully rebased and updated refs/heads/main.

leori@LeonardoPC MINGW64 /d/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1 (main)
$ git push -u origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 6 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 1.41 MiB | 907.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/leoriosmoreno/DesarrolloSistemasInformaticosUTM.git
   1c3e228..4a9ef3d  main -> main
branch 'main' set up to track 'origin/main'.

leori@LeonardoPC MINGW64 /d/ACADEMICO/EDUCACION SUPERIOR/TERCER NIVEL/UTM/TI/5 CICLO/MATERIAS/Desarrollo de Sistemas Informáticos/TAREAS/A1 (main)
$ |
```

GITHUB (En la nube)



Bibliografía

Atlassian. (2024). *Flujo de trabajo Gitflow: Guía completa de ramificación*. Recuperado de https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow

Microsoft Support. (2024). *Crear un diagrama de clases de UML en Visio*. Recuperado de https://support.microsoft.com/es-es/topic/crear-un-diagrama-de-clases-de-uml-en-visio

Refactoring Guru. (s.f.). *Patrones de diseño en el desarrollo de software*. Recuperado de https://refactoring.guru/es/design-patterns

ServiceNow. (2025). *Guía de gestión de incidentes bajo el marco ITIL*. Recuperado de https://www.servicenow.com/latam/products/itsm/what-is-incident-management.html

Splunk. (2024). *Los principios SOLID en la ingeniería de software moderna*. Recuperado de https://www.splunk.com/en_us/blog/learn/solid-design-principle.html

Universidad Técnica de Manabí. (2024). *Compendio Unidad 1: Fundamentos de Ingeniería de Software y Paradigmas de Diseño*. Portoviejo, Ecuador: UTM.